

=====

NULLSPLOIT TECHNICAL DOCUMENTATION

KERNEL FIREWALL & SECURITY OPERATIONS CENTER

TECHNICAL ARCHITECTURE & OPERATIONS MANUAL

=====

CLASSIFICATION:

Internal Use Only

DETAILS:

Version: 1.0

Date: July 2026

Platform: Linux Netfilter/NFQUEUE, Docker

Status: Approved Documentation

1. EXECUTIVE SUMMARY

Nullsploit is a hybrid kernel-userspace network security stack designed to protect Apache web servers from common Layer 4 and Layer 7 network attacks. The system intercepts traffic using Linux's **Netfilter NFQUEUE** framework, inspects packets for known threat signatures, and forwards encrypted telemetry to a centralised SOC dashboard for real-time monitoring and historical analysis.

The stack has four tightly-integrated components: a userspace NFQUEUE firewall daemon (`fw_nfq`), a telemetry agent (`fw_agent`), a PGP-encrypted ingest receiver (`soc_receiver`), and a real-time web dashboard. These are packaged as isolated Docker containers.

1.1 Key Capabilities

Below is a summary of the technical defense mechanisms and

architectural features provided by the Nullsploit stack:

Capability	Description
PHP Shell	Pattern matching against RCE function
Detection	signatures in URI and body
XSS Detection	Case-insensitive scan for cross-site scripting vectors
SYN Flood Mitigation	Sliding-window rate limiting on TCP SYN packets per source IP
Slowloris Detection	Timeout-based tracking of incomplete HTTP header connections
IP Blocklist	CIDR-range lookups in a kernel-side hash table; hot-reloadable
PGP Encryption	All agent-to-SOC telemetry encrypted with GPG public-key crypto
Live Dashboard	SSE live event feed with Chart.js visualisations on port 8443
Secure Auth	bcrypt passwords, HMAC-SHA256 signed session tokens

2. SYSTEM ARCHITECTURE OVERVIEW

The operational flow begins at the packet interception layer and moves up through userspace analysis to secure telemetry transmission and graphical presentation.

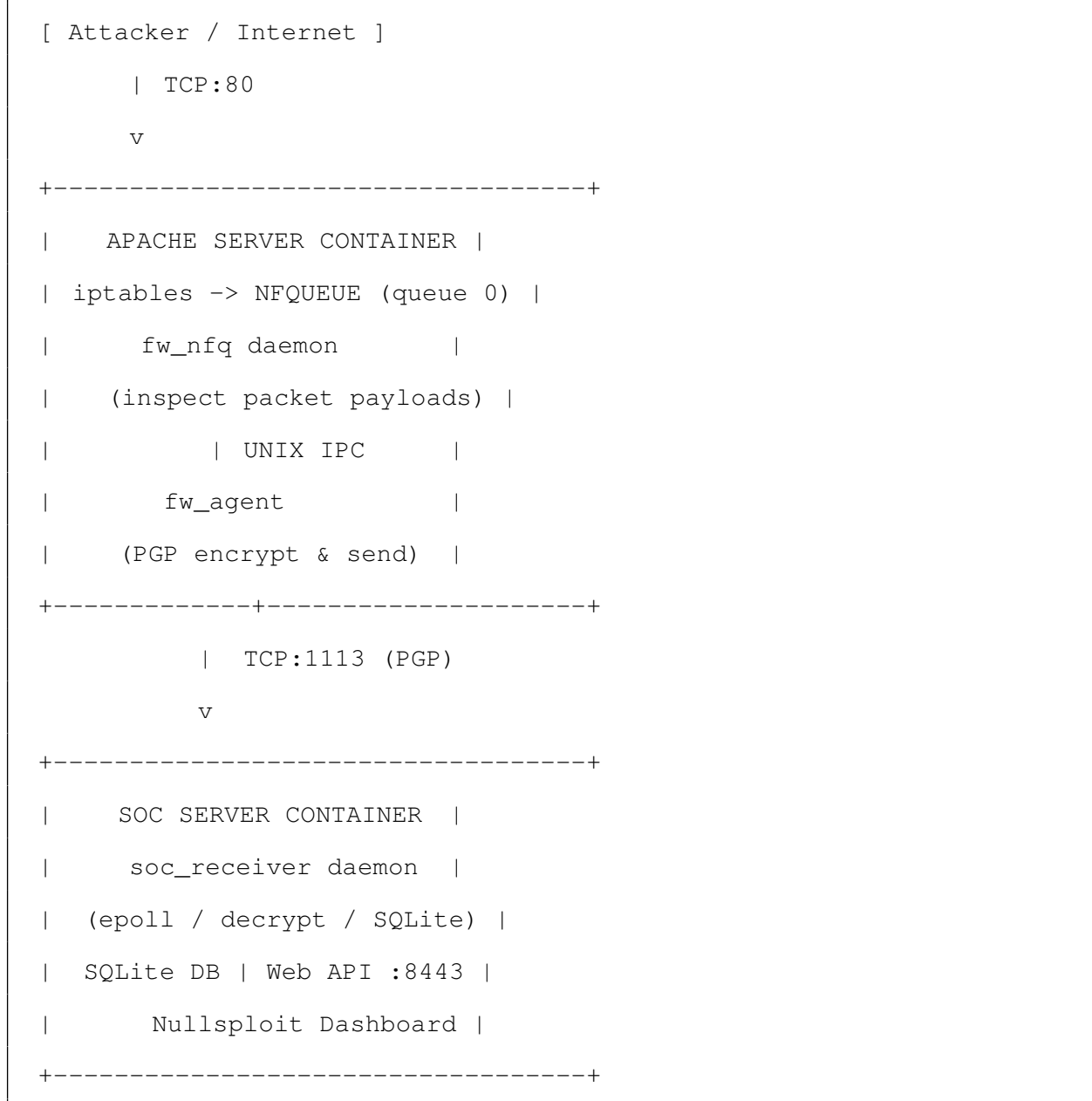
Layer	Component	Role
Network L4/L7	iptables	Redirect TCP/80 to queue 0
	NFQUEUE	for userspace inspection
Userspace	fw_nfq Daemon	Dequeue packets, inspect, issue NF_ACCEPT or NF_DROP
IPC	UNIX Socket	Transfer fw_event structs from fw_nfq to fw_agent
Telemetry	fw_agent	PGP-encrypt; send instantly (WARNING+) or batch (INFO)
Transport	TCP Port 1113	Encrypted channel to the SOC container
Ingest	soc.receiver	epoll loop, decrypts PGP payloads, writes to SQLite
Storage	SQLite	Persistent event store at /var/lib/soc/soc.db
Presentation	Web Dashboard	REST API, SSE live feed, Chart.js on port 8443

2.1 Container Network Topology

Both containers share the Docker bridge network. Service discovery uses container names as DNS hostnames. PGP keys are exchanged via a shared host volume at ./keys.

2.2 Architecture Diagram

The visual flow below details packet redirection and telemetry pathways:



3. COMPONENT: KERNEL FIREWALL (fw.nfq)

The daemon processes incoming frames bound to NFQUEUE queue 0. The raw packet capture is achieved using the iptables rule configured at system initialization:

```
iptables -A INPUT -p tcp --dport 80 -j NFQUEUE --queue-num 0
```

Each packet is evaluated sequentially against the following processing table:

Step	Function	Action on Detection
1	inspect_ip_blocklist()	Immediately issue NF_DROP
2	monitor_tcp_stats()	NF_DROP on SYN flood or Slowloris timeout
3	inspect_http_payload()	NF_DROP on PHP shell or XSS match
4	(none)	Issue NF_ACCEPT --- forward to Apache

3.1 Telemetry Struct (fw_event)

When a signature match or threshold violation is verified, the event parameters are packaged using the following binary structure and transmitted over a local UNIX socket at

/var/run/fw_inspect.sock:

```
struct fw_event {
    uint64_t timestamp;
    uint32_t src_ip;      /* Source IP (network byte order) */
    uint32_t dest_ip;
    uint16_t src_port;
    uint16_t dest_port;
    uint8_t threat_type; /* 1=PHP_SHELL 2=XSS 3=SYN_FLOOD 4=SLOWLORIS
    */
    uint8_t severity;    /* 0=INFO 1=WARNING 2=CRITICAL */
    char    payload_preview[128];
    char    details[256];
};
```

4. COMPONENT: USERSPACE AGENT (fw_agent)

The agent acts as a message broker between the raw socket inspector and the encrypted remote network backend. It implements hot-reloading configurations and real-time alert processing.

4.1 Message Severity and Routing Policy

The agent processes alerts based on severity profiles:

Severity	Value	Examples	Routing
CRITICAL	2	PHP Shell, SYN Flood	Sent immediately (bypasses buffer)
WARNING	1	XSS, Slowloris, Blocklist	Sent immediately (bypasses buffer)
INFO	0	Normal traffic metadata	Buffered; flushed every 5 minutes

4.2 Heartbeat Protocol and Reload Sequence

A keep-alive ping (MSG_TYPE_PING) is sent when the channel is idle for 5 minutes. On receiving a SIGHUP signal, the agent re-reads the configuration file and flushes dynamic entries to the kernel hook:

```
# /etc/fw_inspect/blocklist.txt
# One CIDR entry per line
10.0.0.0/8
192.168.1.50/32
```

5. COMPONENT: SOC RECEIVER (soc_receiver)

The endpoint engine is written in C as a single-threaded server leveraging `epoll(7)`. It processes agent streams, decryption,

database persistence, and websocket/SSE events on distinct ports.

5.1 Decryption Pipeline

Encrypted payloads received from verified agents are processed via a system execution fork using the GnuPG backend wrapper:

```
gpg --decrypt --batch --yes --output /tmp/event.bin /tmp/event.pgp
```

Decrypted telemetry is extracted into memory, verified against buffer limits, and inserted into the SQLite database. Registered web browsers connected to /api/events/live are simultaneously notified using Server-Sent Events (SSE).

6. COMPONENT: WEB DASHBOARD

The front-end user interface is structured as a single-page application (SPA) built using native web components and vanilla CSS. It offers charts, tables, and system logs.

6.1 UI Management Panels

The console layout contains five essential operational sectors:

Panel	Description
Threat Chart	Bar chart: event counts per threat type over time
Active Agents	Lists all nodes with IPs and last-seen timestamps
Live Activity Feed	Real-time scrolling event feed via SSE
Historical Events	Paginated event table from SQLite via /api/events
Stats Overview	Total events, unique IPs, threat breakdown cards

6.2 Application REST API Specifications

Inter-system API communication utilizes the following endpoints:

Method	Endpoint	Description
POST	/auth	Login; returns Set-Cookie: session.token
GET	/api/stats	Event counts grouped by threat.type
GET	/api/agents	List of registered agent nodes
GET	/api/events	Historical events (limit/offset)
GET	/api/events/live	SSE stream of live events

7. PGP ENCRYPTED TRANSPORT PROTOCOL

The server-agent network link is encrypted end-to-end. Public keys are generated on startup and synced using a common Docker storage volume.

7.1 Trust Key Exchange Sequence

The automated handshaking routine is defined below:

Step	Container	Action
1	soc-server	Generate RSA-2048 keypair for soc@soc.local
2	soc-server	Export public key to /etc/keys/soc_pubkey.asc
3	apache-server	Generate RSA-2048 keypair for agent@soc.local
4	apache-server	Import SOC public key from shared volume
5	apache-server	Export agent public key to shared volume
6	soc-server	Key watcher detects agent_pubkey.asc (SHA-256 change) and imports
7	Both	Trust established. Agent encrypts all messages with SOC key.

7.2 Frame Structure

The low-level network header prepended to the PGP ciphertext matches the following protocol struct:

```
struct soc_msg_hdr {  
    uint32_t magic;      /* 0xDEADBEEF - validates message framing */  
    uint8_t msg_type;    /* 0=PING 1=BATCH 2=ALERT */  
    uint8_t agent_uuid[16]; /* Unique per-agent identifier */  
    uint32_t payload_len; /* Length of the following PGP ciphertext */  
};
```

8. THREAT DETECTION SIGNATURES

Payload inspection operates on exact pattern matching targeting the HTTP parameters.

8.1 Remote Code Execution (RCE) Signatures

The stack flags known dangerous PHP executions in request paths and body arguments:

Signature	Severity	Example Attack
<code>eval(</code>	CRITICAL	<code>eval(base64_decode(\$_POST[cmd]))</code>
<code>system(</code>	CRITICAL	<code>GET /?cmd=system(id)</code>
<code>exec(</code>	CRITICAL	<code>exec('cat /etc/passwd')</code>
<code>shell_exec(</code>	CRITICAL	<code>shell_exec('rm -rf /')</code>
<code>passthru(</code>	CRITICAL	<code>passthru(\$_GET['x'])</code>
<code>popen(</code>	CRITICAL	<code>popen('ls', 'r')</code>
<code>proc_open(</code>	CRITICAL	<code>proc_open('whoami', ...)</code>
<code>base64_decode(</code>	CRITICAL	Obfuscated payload delivery

8.2 Cross-Site Scripting (XSS) Patterns

The following patterns are scanned case-insensitively using custom string parsers:

Signature	Attack Vector
<code><script></code>	Direct script injection
<code>javascript:</code>	Anchor href and meta refresh attacks
<code>onerror=</code>	Image/object error event handler injection
<code>onload=</code>	Body/iframe onload event injection
<code><img</code>	Image tag with onerror payload
<code>alert(</code>	Proof-of-concept XSS marker

9. PROTOCOL ATTACK MITIGATION

Layer 4 attacks are tracked and blocked based on stateful monitoring counters.

9.1 SYN Flood Thresholds

Mitigation uses sliding-window tracking parameters:

Parameter	Value	Description
SYN Flood Threshold	100/s	Max SYN/s per source IP before blocking
Window Duration	1 sec	Rate counter reset interval
Block Duration	60s	Packets dropped until pool eviction
Alert throttle	1/window	One SOC alert when threshold is crossed

9.2 Slowloris Constraints

Slow HTTP header streams are detected using connection timers:

Parameter	Value	Description
SLOWLORIS_TIMEOUT	30s	Max time for HTTP headers to complete
Detection basis	Timer	Set on first packet, checked on all subsequent
Action	DROP	Connection dropped; WARNING sent instantly to SOC

10. IP BLOCKLIST HASH TABLE

The blocklist table utilizes an 8-bit hash index with 256 dynamic buckets to store blacklisted hosts and CIDR ranges. Updates are dynamic:

```
# /etc/fw_inspect/blocklist.txt
10.0.0.0/8
192.168.1.50/32
45.33.32.0/24
```

SIGHUP triggers live kernel-level updates without disconnecting current web sockets.

11. AUTHENTICATION & ACCESS CONTROL

Access rules are enforced before requests hit the application layer.

11.1 Allowed Administrator IP Ranges

Only the following network origins can access port 8443:

Range	Purpose
127.0.0.1/32	Localhost administration
172.16.0.0/12	Docker bridge and internal ranges
192.168.0.0/16	LAN subnet for SOC analysts

11.2 Credentials Configuration

Administrators store hashed passwords in a localized flat file.

Successful login returns an HMAC signature token:

```
# /etc/soc/users.conf format
admin:$2b$12$<bcrypt_hash>

# Session cookie format
session_token=username:unix_timestamp:hmac_sha256_hex
```

12. DOCKER DEPLOYMENT SPECS

Container constraints are mapped using the compose configuration schema:

Parameter	apache-server	soc-server
Base Image	debian:bullseye-slim	debian:bullseye-slim
Exposed Ports	80 (HTTP)	1113 (agent), 8443 (web)
Capabilities	NET_ADMIN (iptables)	None
Volumes	./signatures, ./keys	./keys, soc-data (SQLite)
Restart	unless-stopped	unless-stopped

12.1 Manual Operational Triggers

Deployment and control actions are initiated using the commands below:

```
# Full clean start (wipes database and keys)
docker compose down -v
docker compose up -d --build

# Hot-reload blocklist without restart
docker exec apache-2.4.26 kill -HUP $(pidof fw_agent)
```

13. TESTING AND ENGINE VALIDATION

Local validation runs a mock environment using `userspace_compat.h` shims:

Test	Input	Expected	Result
HTTP T1	GET /index.html	ALLOW	PASS
HTTP T2	POST body: eval(data)	DROP - PHP Shell	PASS
HTTP T3	GET /?q=<script>alert(1)	DROP - XSS	PASS
HTTP T4	GET /?name= systematic_eval	ALLOW (no FP)	PASS
Blocklist T1	192.168.1.50 (exact)	DROP	PASS
Blocklist T2	192.168.1.51 (unlisted)	ALLOW	PASS
Blocklist T3	10.250.4.5 in 10.0.0.0/8	DROP	PASS
Blocklist T4	11.0.0.1 outside subnet	ALLOW	PASS
SYN T1	101 SYN/s from one IP	DROP + ALERT	PASS
SYN T2	50 SYN/s from one IP	ALLOW	PASS
Slowloris T1	Headers incomplete after 30s	DROP + ALERT	PASS

14. SYSTEM OPERATIONS GUIDE

Daily configuration rules and references are documented below.

14.1 Login Details

Connection targets for security analysts:

Field	Value
Local URL	http://127.0.0.1:8443/
LAN URL	http://<your-lan-ip>:8443/
Username	admin
Password	password

14.2 Threat Types Reference

Telemetry codes returned by the threat detection system:

threat type	Name	Severity	Description
1	PHP Shell	CRITICAL	RCE via PHP built-in functions
2	XSS	WARNING	Cross-site scripting payload detected
3	SYN Flood	CRITICAL	SYN packet rate exceeds threshold
4	Slowloris	WARNING	Incomplete HTTP header connection timeout
5	IP Blocklist	WARNING	Connection from known malicious IP

14.3 Maintenance Command Cheat-Sheet

Operational utilities for log monitoring and manual configuration:

```
# Firewall verdicts and alerts
docker exec apache-2.4.26 cat /var/log/fw_nfq.log

# Agent routing and SOC transmissions
docker exec apache-2.4.26 cat /var/log/fw_agent.log

# SOC receiver log
docker logs soc-dashboard

# Query events via API
curl -b cookies.txt http://127.0.0.1:8443/api/events
```



```
# Add IPs to blocklist then hot-reload  
notepad signatures/blocklist.txt  
docker exec apache-2.4.26 kill -HUP $(pidof fw_agent)
```

=====

END OF REPORT

=====